DESENVOLVIMENTO WEB COM .NET

ANÁLISE E DESENVOLVIMENTO DE SISTEMAS

RELATÓRIO DE AULAS PRÁTICAS

Relatório 7 – Logging, métricas, tracing, Health Checks, resiliência, rate limiting e output caching

Nome: _____

RA: _____

Polo de matrícula: _____

Local da realização da Aula Prática: _____

Ano da postagem: 2026

Instituto de Ciências Exatas e Tecnologia	**Disciplina:** Desenvolvimento Web com .NET **Título da Aula:** Logging, métricas, tracing, Health Checks, resiliência, rate limiting e output caching	**Relatório 7**

Unidade 7 – Observabilidade e resiliência

Objetivo: Instrumentar a aplicação e aplicar controles de resiliência e capacidade, produzindo evidências úteis para diagnóstico.

1. Identificação e ambiente

Repositório: https://github.com/maiaralopes-dev/projetos (branch `ccr-836bb896-v5ejij`, pasta `PortalTarefas/`); commit `6a1dece`. OpenTelemetry .NET 1.20 (Extensions.Hosting, instrumentações AspNetCore, Http e SqlClient, exportadores Console e OTLP), `Microsoft.Extensions.Http.Resilience` 10.11 (Polly v8), Health Checks com `AddDbContextCheck`. Painel local: **Aspire Dashboard 9.5** em contêiner recebendo OTLP em `localhost:4317`. Dependência externa simulada: `scripts/dependencia-calendario.js` (Node), com modos `ok`, `falha503`, `falha400`, `lento` e `fora`.

2. Resumo teórico aplicado ao projeto

Logs, métricas e traces. *Logs* são eventos discretos com contexto (o que aconteceu); *métricas* são números agregados ao longo do tempo (quanto/quão rápido); *traces* mostram o caminho de uma requisição em *spans* encadeados (onde o tempo foi gasto). O **identificador de correlação** liga os três: o `X-Correlation-Id` está no escopo de todos os logs da requisição, no Problem Details devolvido ao usuário e é propagado à dependência; o `TraceId` do OpenTelemetry também aparece nos logs JSON.

Logging estruturado. Mensagens com *templates* (`{TarefaId}`) geram campos pesquisáveis; em produção o console usa o formatador JSON com escopos. Níveis: `Debug` para entrada/saída de middleware, `Information` para operações, `Warning` para falhas tratadas (dependência, 429), `Error` para exceções. É proibido registrar senha, cookie, token, connection string ou corpo de requisição; dados pessoais só quando necessários.

Tipos de métrica e cardinalidade. *Counter* só cresce (tarefas criadas); *gauge* é um valor instantâneo (requisições ativas); *histograma* distribui medidas em faixas (duração → p50/p90/p99). Cada combinação de dimensões vira uma série; por isso as dimensões são de **cardinalidade controlada**: `prioridade` (3 valores) e `resultado` (4) — nunca id de tarefa, usuário ou URL completa.

Tracing distribuído. Cada operação é um *span* com pai; o contexto viaja no cabeçalho W3C `traceparent` (visível no log da dependência). A instrumentação cria spans para a requisição HTTP, cada comando SQL e cada chamada `HttpClient`; o span próprio `tarefa.concluir` agrupa os comandos da transação.

Liveness × readiness. *Liveness* responde “o processo está vivo?” — não consulta dependências, senão um banco fora faria o orquestrador reiniciar contêineres sem resolver nada. *Readiness* responde “posso receber tráfego?” e verifica banco e calendário. A

resposta pública traz só `Healthy/Unhealthy`; nomes, descrições e durações ficam em `/health/detalhes`, restrito ao Administrador.

Retry, timeout e circuit breaker. *Timeout* limita a espera; *retry* repete falhas **transitórias** (5xx, 408, 429, erro de rede) com backoff exponencial e jitter; *circuit breaker* para de chamar uma dependência que está falhando, devolvendo erro imediato até um teste de recuperação (meio-aberto). Repetir operações **não idempotentes** (POST de pagamento, por exemplo) pode duplicar efeitos — por isso o retry foi desativado para métodos inseguros (`DisableForUnsafeHttpMethods`).

Rate limiting × output caching. Rate limiting **protege** a capacidade limitando requisições por usuário/IP (429 + `Retry-After`); output caching **melhora desempenho** guardando a resposta inteira de uma leitura pública e repetindo-a sem executar o endpoint. São independentes: uma resposta em cache ainda conta para o limite.

3. Código-fonte e evidências

Tabela 1 – Instrumentação e controles configurados

Sinal / controle	Implementação
Logs	JSON no console (prod.) com escopo `CorrelationId`; OTLP para o painel; log de início/resultado/duração por requisição e operação
Métrica de negócio	`portal.tarefas.criadas{prioridade}`, `portal.tarefas.concluidas` (Meter `PortalTarefas`)
Métrica técnica	Histograma `portal.dependencia.calendario.duracao{resultado}` (ms) + métricas nativas `http.server.request.duration` e de rate limiting
Traces	AspNetCore + HttpClient + SqlClient + span `tarefa.concluir` (tag `tarefa.id`)
Health	`/health/live` (sem verificações), `/health/ready` (banco + calendário), `/health/detalhes` (Admin)
Resiliência	Timeout total 8 s › retry 3× exponencial (só GET) › circuit breaker (50 %, mín. 4, 30 s, pausa 15 s) › timeout por tentativa 2 s
Rate limiting	Janela fixa: `api` 100/10 s por usuário; `publica` 30/10 s por IP; `login` 10/min por IP
Output cache	Política `feriados`: 60 s, varia por `ano`, tag para invalidação (`POST /api/v2/calendario/cache/invalidar`, Admin)

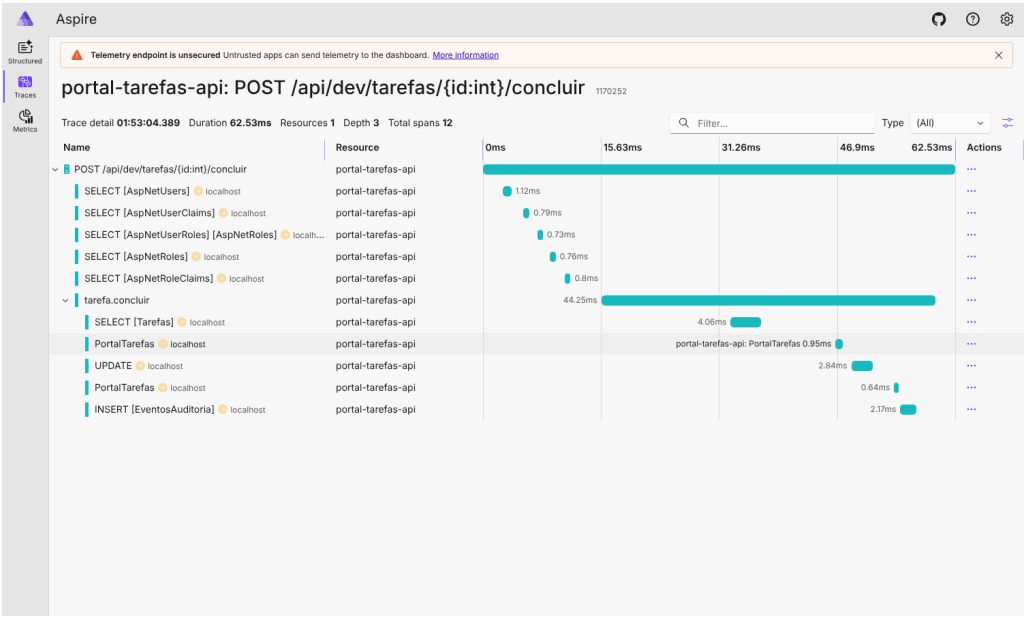


Figura 1 – Trace no painel local: requisição → span customizado tarefa.concluir → SELECT, UPDATE e INSERT da transação

Erro localizado pela correlação: a requisição com `X-Correlation-Id: demo-erro-7` devolveu 500 genérico; filtrando os logs por esse valor aparecem a entrada, as consultas, o erro do `TratadorExcecoesGlobal` e a saída, todos com o mesmo `TraceId`; a busca por senha, cookie ou token no log retornou **0** ocorrências (`01-correlacao-erro.txt`).

4. Testes e validações obrigatórias

Tabela 2 – Testes obrigatórios

Caso exigido	Resultado observado	Evidência
Erro identificável por correlação sem segredo	500 com `correlationId: demo-erro-7`; todas as linhas de log da requisição com o mesmo id e `TraceId`; 0 segredos	`01-correlacao-erro.txt`; Fig. B2
Dependência fora: readiness × liveness	Calendário `fora`: `/health/ready` 503 e `/health/live` 200; detalhes (Admin) apontam `calendario` Unhealthy	`03-health.txt`; 2 testes
Retry só em falha transitória e abertura do circuito	400 → 1 requisição (sem retry), 502. 503 → 3 tentativas; a 3ª retentativa já encontrou o circuito aberto → 503; chamadas seguintes falham rápido sem tocar a dependência; após 16 s, sucesso	`04-resiliencia.txt`; Fig. B4
429 após exceder o limite	Política pública reduzida a 10/10 s: 10 × 200 e 4 × 429 com `Retry-After: 10`; teste por usuário (5 × 200, 2 × 429, outro usuário 200)	`06-rate-limiting.txt`; teste
Cache em endpoint público não personalizado	5 GET anônimos → 1 chamada à dependência; 2 GET autenticados → 2 chamadas (não usam cache); após invalidar → nova chamada	`05-cache.txt`

Carga controlada local (autocannon, 10 conexões, 10 s, `/health/live`): ~11,9 mil req/s em média (119 mil requisições), latência p97,5 de 3 ms. O teste rodou em contêiner de desenvolvimento com 4 vCPU, build Debug e banco no mesmo host; o número **não** indica capacidade de produção — serve apenas para observar o comportamento e as métricas sob carga.

5. Aprimoramentos

- **Painel local** com métricas, traces e logs (Aspire Dashboard via OTLP) — Figuras 1 e B1–B4.
- **Teste automatizado da política de rate limiting** (`Excesso_de_requisicoes_retorna_429_com_retry_after_por_usuario`) com limites reduzidos por configuração em uma fábrica de teste própria, além de testes de health checks.

Tabela 3 – Alertas propostos (conceituais)

Alerta conceitual	Limiar	Ação esperada
Erros do servidor	5xx > 2 % de `http.server.request.duration` por 5 min	Abrir incidente; filtrar traces/logs pelo `correlationId`; avaliar rollback da última versão
Dependência degradada	`portal.dependencia.calendario.duracao{resultado=circuito_aberto}` > 0 por 5 min	Avisar equipe; checar status do serviço; comunicar indisponibilidade parcial
Instância não pronta	`/health/ready` ≠ 200 por 2 min	Retirar do balanceador; verificar banco e calendário
Abuso ou cota mal dimensionada	Taxa de 429 > 5 % por 10 min	Investigar origem; ajustar limites com base em dados

Referências

MICROSOFT. **Logging in .NET and ASP.NET Core**. Microsoft Learn, 2025. Disponível em: https://learn.microsoft.com/aspnet/core/fundamentals/logging/. Acesso em: 9 nov. 2026.

MICROSOFT. **.NET observability with OpenTelemetry**. Microsoft Learn, 2025. Disponível em: https://learn.microsoft.com/dotnet/core/diagnostics/observability-with-otel. Acesso em: 9 nov. 2026.

MICROSOFT. **Health checks in ASP.NET Core**. Microsoft Learn, 2025. Disponível em: https://learn.microsoft.com/aspnet/core/host-and-deploy/health-checks. Acesso em: 9 nov. 2026.

MICROSOFT. **Build resilient HTTP apps: Key development patterns**. Microsoft Learn, 2025. Disponível em: https://learn.microsoft.com/dotnet/core/resilience/http-resilience. Acesso em: 9 nov. 2026.

MICROSOFT. **Rate limiting middleware in ASP.NET Core**. Microsoft Learn, 2025. Disponível em: https://learn.microsoft.com/aspnet/core/performance/rate-limit. Acesso em: 9 nov. 2026.

MICROSOFT. **Output caching middleware in ASP.NET Core**. Microsoft Learn, 2025. Disponível em: https://learn.microsoft.com/aspnet/core/performance/caching/output. Acesso em: 9 nov. 2026.

OPENTELEMETRY. **Observability primer** e **Semantic conventions for HTTP metrics**. opentelemetry.io, 2025. Disponível em: https://opentelemetry.io/docs/concepts/observability-primer/. Acesso em: 9 nov. 2026.

W3C. **Trace Context**. W3C Recommendation, 2021. Disponível em: https://www.w3.org/TR/trace-context/. Acesso em: 9 nov. 2026.

Apêndice A – Código-fonte integral

Arquivo: `src/PortalTarefas.Web/Observabilidade/Telemetria.cs` (commit `6a1dece`)

```
using System.Diagnostics;
using System.Diagnostics.Metrics;

namespace PortalTarefas.Web.Observabilidade;

/// <summary>Fontes próprias de traces e métricas do Portal de Tarefas.</summary>
public sealed class Telemetria
{
    public const string Nome = "PortalTarefas";
    public static readonly ActivitySource Fonte = new(Nome);

    private readonly Counter<long> _tarefasCriadas;
    private readonly Counter<long> _tarefasConcluidas;
    private readonly Histogram<double> _duracaoCalendario;

    public Telemetria(IMeterFactory fabrica)
    {
        var meter = fabrica.Create(Nome);
        // Métrica de negócio: dimensão "prioridade" tem só 3 valores possíveis (cardinalidade controlada).
        _tarefasCriadas = meter.CreateCounter<long>("portal.tarefas.criadas", "{tarefa}", "Tarefas cadastradas");
        _tarefasConcluidas = meter.CreateCounter<long>("portal.tarefas.concluidas", "{tarefa}", "Tarefas
concluídas");
        // Métrica técnica: duração das chamadas à dependência externa, por resultado
(sucesso/falha/circuito_aberto/timeout).
        _duracaoCalendario = meter.CreateHistogram<double>("portal.dependencia.calendario.duracao", "ms",
            "Duração das chamadas ao serviço de calendário (inclui tentativas)");
    }

    public void TarefaCriada(string prioridade) => _tarefasCriadas.Add(1, new KeyValuePair<string, object?
>("prioridade", prioridade));
    public void TarefaConcluida() => _tarefasConcluidas.Add(1);
    public void ChamadaCalendario(double ms, string resultado) =>
        _duracaoCalendario.Record(ms, new KeyValuePair<string, object?>("resultado", resultado));
}
```

Arquivo: `src/PortalTarefas.Web/Observabilidade/ObservabilidadeExtensions.cs` (commit `6a1dece`)

```
using OpenTelemetry;
using OpenTelemetry.Logs;
using OpenTelemetry.Metrics;
using OpenTelemetry.Resources;
using OpenTelemetry.Trace;

namespace PortalTarefas.Web.Observabilidade;

public static class ObservabilidadeExtensions
{
    /// <summary>
    /// Configura OpenTelemetry (traces, métricas e logs). Exportadores:
    ///  - console, quando Telemetria:Console = true (evidência local);
    ///  - OTLP, quando OTEL_EXPORTER_OTLP_ENDPOINT estiver definido (coletor/painel local, ex.: Aspire
Dashboard).
    /// </summary>
    public static WebApplicationBuilder AddObservabilidade(this WebApplicationBuilder builder)
    {
        builder.Services.AddSingleton<Telemetria>();
        var console = builder.Configuration.GetValue<bool>("Telemetria:Console");
        var otlp = !string.IsNullOrWhiteSpace(builder.Configuration["OTEL_EXPORTER_OTLP_ENDPOINT"]);

        var otel = builder.Services.AddOpenTelemetry()
            .ConfigureResource(r => r.AddService("portal-tarefas-api",
                serviceVersion: builder.Configuration["APP_VERSION"] ?? "dev",
                serviceInstanceId: Environment.MachineName))
            .WithTracing(t =>
            {
                t.AddSource(Telemetria.Nome)
                 .AddAspNetCoreInstrumentation(o =>
                     // Sondas de saúde e ping geram ruído e não ajudam no diagnóstico.
                     o.Filter = ctx => !ctx.Request.Path.StartsWithSegments("/health") && !
ctx.Request.Path.StartsWithSegments("/ping"))
                 .AddHttpClientInstrumentation()
                 .AddSqlClientInstrumentation();
                if (console) t.AddConsoleExporter();
            })
            .WithMetrics(m =>
            {
                m.AddMeter(Telemetria.Nome)
                 .AddAspNetCoreInstrumentation()           // http.server.request.duration etc.
                 .AddHttpClientInstrumentation()
                 .AddMeter("Microsoft.AspNetCore.RateLimiting", "Microsoft.AspNetCore.Diagnostics", "Polly");
                if (console) m.AddConsoleExporter((_, r) =>
r.PeriodicExportingMetricReaderOptions.ExportIntervalMilliseconds = 15000);
            })
            .WithLogging(_ => { }, o => { o.IncludeScopes = true; o.IncludeFormattedMessage = true; });

        if (otlp) otel.UseOtlpExporter();   // traces + métricas + logs para o mesmo endpoint OTLP
```

```
        return builder;
    }
}
```

Arquivo: `src/PortalTarefas.Web/Servicos/CalendarioCliente.cs` (commit `6a1dece`)

```
using System.Diagnostics;
using Polly.CircuitBreaker;
using Polly.Timeout;
using PortalTarefas.Web.Observabilidade;

namespace PortalTarefas.Web.Servicos;

public sealed record Feriado(DateOnly Data, string Nome);

/// <summary>Resultado sem exceção, para o endpoint decidir o status HTTP.</summary>
public sealed record ResultadoCalendario(IReadOnlyList<Feriado>? Feriados, string Resultado);

/// <summary>
/// Cliente tipado da dependência externa "serviço de calendário". O HttpClient recebe, por configuração,
/// timeout por tentativa, retry só para falhas transitórias em GET e circuit breaker (ver Program.cs).
/// </summary>
public sealed class CalendarioCliente(HttpClient http, Telemetria telemetria, ILogger<CalendarioCliente> logger)
{
    public async Task<ResultadoCalendario> ObterFeriadosAsync(int ano, CancellationToken ct)
    {
        var inicio = Stopwatch.GetTimestamp();
        var resultado = "falha";
        try
        {
            var feriados = await http.GetFromJsonAsync<List<Feriado>>($"feriados/{ano}", ct) ?? [];
            resultado = "sucesso";
            return new(feriados, resultado);
        }
        catch (BrokenCircuitException)
        {
            resultado = "circuito_aberto";
            logger.LogWarning("Calendário: circuito aberto; chamada recusada sem acessar a dependência");
        }
        catch (TimeoutRejectedException)
        {
            resultado = "timeout";
            logger.LogWarning("Calendário: tempo limite total excedido");
        }
        catch (HttpRequestException ex)
        {
            logger.LogWarning("Calendário: falha {Motivo} após as tentativas configuradas",
                ex.StatusCode is { } s ? $"HTTP {(int)s}" : ex.HttpRequestError.ToString());
        }
        finally
        {
            var ms = Stopwatch.GetElapsedTime(inicio).TotalMilliseconds;
            telemetria.ChamadaCalendario(ms, resultado);
            logger.LogInformation("Calendário: ano {Ano} -> {Resultado} em {DuracaoMs:F0} ms", ano, resultado,
ms);
        }
        return new(null, resultado);
    }
}

/// <summary>Propaga o identificador de correlação para a dependência (o traceparent W3C é propagado pelo
OpenTelemetry).</summary>
public sealed class PropagarCorrelacaoHandler(IHttpContextAccessor acessor) : DelegatingHandler
{
    protected override Task<HttpResponseMessage> SendAsync(HttpRequestMessage request, CancellationToken ct)
    {
        if (acessor.HttpContext?.Items[Infraestrutura.CorrelationIdMiddleware.ChaveItem] is string id)
            request.Headers.TryAddWithoutValidation(Infraestrutura.CorrelationIdMiddleware.NomeCabecalho, id);
        return base.SendAsync(request, ct);
    }
}

/// <summary>Readiness: a dependência essencial responde? Usa um cliente sem retry, com timeout curto.</summary>
public sealed class CalendarioHealthCheck(IHttpClientFactory fabrica) :
Microsoft.Extensions.Diagnostics.HealthChecks.IHealthCheck
{
    public async Task<Microsoft.Extensions.Diagnostics.HealthChecks.HealthCheckResult> CheckHealthAsync(
        Microsoft.Extensions.Diagnostics.HealthChecks.HealthCheckContext context, CancellationToken ct = default)
    {
        try
        {
            using var resp = await fabrica.CreateClient("calendario-saude").GetAsync("health", ct);
            return resp.IsSuccessStatusCode
                ? Microsoft.Extensions.Diagnostics.HealthChecks.HealthCheckResult.Healthy("Serviço de calendário
respondeu.")
                : Microsoft.Extensions.Diagnostics.HealthChecks.HealthCheckResult.Unhealthy($"Serviço de
calendário respondeu {(int)resp.StatusCode}.");
        }
        catch (Exception ex) when (ex is HttpRequestException or TaskCanceledException)
        {
            return Microsoft.Extensions.Diagnostics.HealthChecks.HealthCheckResult.Unhealthy("Serviço de
```

```
calendário inacessível.");
        }
    }
}
```

Arquivo: `src/PortalTarefas.Web/Endpoints/OperacaoEndpoints.cs` (commit `6a1dece`)

```
using System.Text.Json;
using Microsoft.AspNetCore.Diagnostics.HealthChecks;
using Microsoft.AspNetCore.OutputCaching;
using Microsoft.Extensions.Diagnostics.HealthChecks;
using PortalTarefas.Web.Seguranca;
using PortalTarefas.Web.Servicos;

namespace PortalTarefas.Web.Endpoints;

public static class OperacaoEndpoints
{
    public static IEndpointRouteBuilder MapOperacao(this IEndpointRouteBuilder app)
    {
        // Liveness: o processo responde? Não consulta dependências (reiniciar o contêiner não resolveria o banco
fora).
        app.MapHealthChecks("/health/live", new HealthCheckOptions { Predicate = _ => false }).AllowAnonymous();

        // Readiness: pode receber tráfego? Verifica banco e dependência essencial. Resposta pública só com o
status.
        app.MapHealthChecks("/health/ready", new HealthCheckOptions { Predicate = c =>
c.Tags.Contains("ready") }).AllowAnonymous();

        // Detalhes (nome de cada verificação, status e duração) somente para administradores.
        app.MapHealthChecks("/health/detalhes", new HealthCheckOptions
        {
            Predicate = c => c.Tags.Contains("ready"),
            ResponseWriter = (ctx, rel) => ctx.Response.WriteAsJsonAsync(new
            {
                status = rel.Status.ToString(),
                duracaoMs = rel.TotalDuration.TotalMilliseconds,
                verificacoes = rel.Entries.Select(e => new { nome = e.Key, status = e.Value.Status.ToString(),
e.Value.Description, duracaoMs = e.Value.Duration.TotalMilliseconds })
            })
        }).RequireAuthorization(Politicas.ExcluirTarefas);

        // Leitura pública e não personalizada: candidata a output caching.
        var cal = app.MapGroup("/api/v2/calendario").WithTags("Calendário");
        cal.MapGet("/feriados/{ano:int}", async (int ano, CalendarioCliente cliente, CancellationToken ct) =>
            {
                if (ano is < 2000 or > 2100)
                    return Results.ValidationProblem(new Dictionary<string, string[]> { ["ano"] = ["Ano entre
2000 e 2100."] });
                var r = await cliente.ObterFeriadosAsync(ano, ct);
                return r.Resultado switch
                {
                    "sucesso" => Results.Ok(r.Feriados),
                    "circuito_aberto" => Results.Problem(statusCode: 503, title: "Serviço de calendário
temporariamente indisponível.",
                        detail: "Proteção por circuit breaker ativa; tente novamente em instantes."),
                    "timeout" => Results.Problem(statusCode: 504, title: "Serviço de calendário não respondeu a
tempo."),
                    _ => Results.Problem(statusCode: 502, title: "Falha ao consultar o serviço de calendário.")
                };
            })
            .AllowAnonymous()
            .CacheOutput("feriados")
            .RequireRateLimiting("publica")
            .WithName("ObterFeriados").WithSummary("Feriados nacionais do ano (público, em cache por 60 s).");

        cal.MapPost("/cache/invalidar", async (IOutputCacheStore cache, CancellationToken ct) =>
            {
                await cache.EvictByTagAsync("feriados", ct);
                return Results.NoContent();
            })
            .RequireAuthorization(Politicas.ExcluirTarefas)
            .AddEndpointFilter<AntiforgeryApiFilter>()
            .WithName("InvalidarCacheFeriados").WithSummary("Invalida o cache de feriados (Administrador).");
        return app;
    }
}
```

Arquivo: `src/PortalTarefas.Web/appsettings.json` (commit `6a1dece`)

```
{
  "Logging": {
    "LogLevel": {
      "Default": "Information",
      "Microsoft.AspNetCore": "Warning"
    },
    "Console": {
      "FormatterName": "json",
      "FormatterOptions": {
        "IncludeScopes": true,
```

```
        "TimestampFormat": "yyyy-MM-ddTHH:mm:ss.fffZ",
        "UseUtcTimestamp": true,
        "JsonWriterOptions": {
          "Indented": false
        }
      }
    }
  },
  "AllowedHosts": "*",
  "Cors": {
    "Origens": []
  },
  "Dependencias": {
    "Calendario": {
      "Url": "http://localhost:5099/"
    }
  },
  "RateLimit": {
    "Api": {
      "Limite": 100,
      "JanelaSegundos": 10
    },
    "Publica": {
      "Limite": 30,
      "JanelaSegundos": 10
    },
    "Login": {
      "Limite": 10
    }
  },
  "Telemetria": {
    "Console": false
  }
}
```

Arquivo: `scripts/dependencia-calendario.js` (commit `6a1dece`)

```
// Serviço de calendário FALSO (dependência externa simulada) para demonstrar health checks e resiliência.
// GET /health                -> 200 (ou 503 no modo "fora")
// GET /feriados/:ano         -> lista de feriados, conforme o modo
// POST /controle?modo=...    -> ok | falha503 | falha400 | lento | fora
// GET /estatisticas          -> total de requisições recebidas em /feriados
const http = require('http');
const porta = Number(process.env.PORTA || 5099);
let modo = 'ok', recebidas = 0;
const feriados = (ano) => [
  { data: `${ano}-01-01`, nome: 'Confraternização Universal' }, { data: `${ano}-04-21`, nome: 'Tiradentes' },
  { data: `${ano}-05-01`, nome: 'Dia do Trabalho' }, { data: `${ano}-09-07`, nome: 'Independência do Brasil' },
  { data: `${ano}-10-12`, nome: 'Nossa Senhora Aparecida' }, { data: `${ano}-11-02`, nome: 'Finados' },
  { data: `${ano}-11-15`, nome: 'Proclamação da República' }, { data: `${ano}-11-20`, nome: 'Dia Nacional de
Zumbi e da Consciência Negra' },
  { data: `${ano}-12-25`, nome: 'Natal' } ];
http.createServer((req, res) => {
  const url = new URL(req.url, 'http://x');
  const json = (status, corpo) => { res.writeHead(status, { 'Content-Type': 'application/json' });
res.end(JSON.stringify(corpo)); };
  if (url.pathname === '/controle') { modo = url.searchParams.get('modo') || 'ok'; recebidas = 0;
console.log(`[calendario] modo=${modo}`); return json(200, { modo }); }
  if (url.pathname === '/estatisticas') return json(200, { modo, recebidas });
  if (url.pathname === '/health') return modo === 'fora' ? json(503, { status: 'fora' }) : json(200, { status:
'ok' });
  const m = url.pathname.match(/^\/feriados\/(\d{4})$/);
  if (!m) return json(404, {});
  recebidas++;
  console.log(`[calendario] #${recebidas} ${req.method} ${url.pathname} modo=${modo} correlation=$
{req.headers['x-correlation-id'] || '-'} traceparent=${req.headers['traceparent'] || '-'}`);
  if (modo === 'falha503') return json(503, { erro: 'indisponível' });
  if (modo === 'falha400') return json(400, { erro: 'requisição inválida' });
  if (modo === 'fora') return req.socket.destroy();          // conexão derrubada
  if (modo === 'lento') return setTimeout(() => json(200, feriados(m[1])), 5000);
  json(200, feriados(m[1]));
}).listen(porta, () => console.log(`[calendario] ouvindo em http://localhost:${porta}`));
```

Program.cs (trecho): HttpClient resiliente, health checks, rate limiting e output cache

```
// ---------- Dependência externa com resiliência (unidade 7) ----------
var urlCalendario = builder.Configuration["Dependencias:Calendario:Url"] ?? "http://localhost:5099/";
builder.Services.AddHttpContextAccessor();
builder.Services.AddTransient<PropagarCorrelacaoHandler>();
builder.Services.AddHttpClient<CalendarioCliente>(c => c.BaseAddress = new Uri(urlCalendario))
    .AddHttpMessageHandler<PropagarCorrelacaoHandler>()
    .AddResilienceHandler("calendario", p =>
    {
        // Ordem: tempo total > tentativas > disjuntor > tempo por tentativa (o mais interno).
        p.AddTimeout(TimeSpan.FromSeconds(8));
        var retry = new HttpRetryStrategyOptions
        {
            MaxRetryAttempts = 3,                       // finito
            BackoffType = DelayBackoffType.Exponential, // 200 ms, 400 ms, 800 ms (+ jitter)
            Delay = TimeSpan.FromMilliseconds(200),
```

```
            UseJitter = true,
            // Padrão: só falhas transitórias (5xx, 408, 429, erro de rede, timeout). 400/404 não são repetidos.
        };
        retry.DisableForUnsafeHttpMethods();            // nunca repete POST/PUT/PATCH/DELETE
        p.AddRetry(retry);
        p.AddCircuitBreaker(new HttpCircuitBreakerStrategyOptions
        {
            FailureRatio = 0.5,
            MinimumThroughput = 4,
            SamplingDuration = TimeSpan.FromSeconds(30),
            BreakDuration = TimeSpan.FromSeconds(15),
        });
        p.AddTimeout(TimeSpan.FromSeconds(2));
    });
builder.Services.AddHttpClient("calendario-saude", c => { c.BaseAddress = new Uri(urlCalendario); c.Timeout =
TimeSpan.FromSeconds(2); });

builder.Services.AddHealthChecks()
    .AddDbContextCheck<PortalTarefasDbContext>("banco", tags: ["ready"])
    .AddCheck<CalendarioHealthCheck>("calendario", tags: ["ready"]);

// ---------- Rate limiting e output caching (unidade 7) ----------
var limites = builder.Configuration.GetSection("RateLimit");
builder.Services.AddRateLimiter(o =>
{
    o.RejectionStatusCode = StatusCodes.Status429TooManyRequests;
    o.OnRejected = async (ctx, ct) =>
    {
        if (ctx.Lease.TryGetMetadata(MetadataName.RetryAfter, out var espera))
            ctx.HttpContext.Response.Headers.RetryAfter = ((int)Math.Ceiling(espera.TotalSeconds)).ToString();
        ctx.HttpContext.RequestServices.GetRequiredService<ILoggerFactory>().CreateLogger("RateLimiting")
            .LogWarning("Limite de requisições excedido em {Caminho}", ctx.HttpContext.Request.Path);
        await ctx.HttpContext.RequestServices.GetRequiredService<IProblemDetailsService>().WriteAsync(new()
        {
            HttpContext = ctx.HttpContext,
            ProblemDetails = { Status = 429, Title = "Muitas requisições.", Detail = "Aguarde o tempo indicado em
Retry-After." }
        });
    };
    // Usuário autenticado: cota por usuário; anônimo: cota por IP.
    static string Chave(HttpContext c) => c.User.Identity?.IsAuthenticated == true
        ? "u:" + c.User.Identity.Name : "ip:" + c.Connection.RemoteIpAddress;
    o.AddPolicy("api", c => RateLimitPartition.GetFixedWindowLimiter(Chave(c), _ => new
FixedWindowRateLimiterOptions
    {
        PermitLimit = limites.GetValue("Api:Limite", 100), Window =
TimeSpan.FromSeconds(limites.GetValue("Api:JanelaSegundos", 10)), QueueLimit = 0
    }));
    o.AddPolicy("publica", c => RateLimitPartition.GetFixedWindowLimiter(Chave(c), _ => new
FixedWindowRateLimiterOptions
    {
        PermitLimit = limites.GetValue("Publica:Limite", 30), Window =
TimeSpan.FromSeconds(limites.GetValue("Publica:JanelaSegundos", 10)), QueueLimit = 0
    }));
    o.AddPolicy("login", c => RateLimitPartition.GetFixedWindowLimiter("ip:" + c.Connection.RemoteIpAddress, _ =>
new FixedWindowRateLimiterOptions
    {
        PermitLimit = limites.GetValue("Login:Limite", 10), Window = TimeSpan.FromMinutes(1), QueueLimit = 0
    }));
});
builder.Services.AddOutputCache(o =>
    // Política base: só GET/HEAD 200 de requisições anônimas e sem cookie de resposta são armazenados.
    o.AddPolicy("feriados", b => b.Expire(TimeSpan.FromSeconds(60)).SetVaryByRouteValue("ano").Tag("feriados")));
```

Apêndice B – Evidências

Resiliência (resumo)

```
### Dependência responde 400 (não transitória) – sem retry
HTTP/2 502
    "title": "Falha ao consultar o serviço de calendário.",
Requisições recebidas pela dependência: {"modo":"falha400","recebidas":1}
### Dependência responde 503 (transitória) – 1 tentativa + 3 retries
HTTP/2 503
    "title": "Serviço de calendário temporariamente indisponível.",
Requisições recebidas pela dependência: {"modo":"falha503","recebidas":3}
### Nova chamada logo em seguida – circuito ABERTO, falha rápida
HTTP/2 503
    "title": "Serviço de calendário temporariamente indisponível.",
### Mais uma – circuito continua aberto
HTTP/2 503
    "title": "Serviço de calendário temporariamente indisponível.",
Requisições recebidas pela dependência (não aumentou com o circuito aberto):
{"modo":"falha503","recebidas":3}
-- dependência restabelecida; aguardando 16 s (BreakDuration = 15 s)
### Após a pausa: circuito meio-aberto -> chamada de teste com sucesso -> fechado
HTTP/2 200
```

```
Requisições recebidas pela dependência: {"modo":"ok","recebidas":1}
```

Log da dependência: tentativas recebidas, correlation id e traceparent propagados

```
[calendario] #1 GET /feriados/2030 modo=falha400 correlation=463c8de13c284341a5932ff278b17d70 traceparent=00-
ec4dc4c5444652e8d929e140d5d19564-e7ad1e33b54a396a-01
[calendario] #1 GET /feriados/2031 modo=falha503 correlation=6780ca67c4f14ee4b0302095e1a0de0f traceparent=00-
69eca0f68ebf4d94bddbc0a4dcd8b8e2-87d3eff17fa89a95-01
[calendario] #2 GET /feriados/2031 modo=falha503 correlation=6780ca67c4f14ee4b0302095e1a0de0f traceparent=00-
69eca0f68ebf4d94bddbc0a4dcd8b8e2-715bfe756d82802a-01
[calendario] #3 GET /feriados/2031 modo=falha503 correlation=6780ca67c4f14ee4b0302095e1a0de0f traceparent=00-
69eca0f68ebf4d94bddbc0a4dcd8b8e2-1e0b53b67d68d4dc-01
[calendario] #1 GET /feriados/2034 modo=ok correlation=83bd677f54ff40a38312e244785ec78c traceparent=00-
6e1314046fac4ff6313cc4cd680af365-8701a25f1b738373-01
[calendario] #1 GET /feriados/2026 modo=ok correlation=55973cf370fb4d14b49ab57afae79440 traceparent=00-
df651329a9f9a7a2dcc848812cd0ef8a-a226c914d3b1505a-01
[calendario] #1 GET /feriados/2026 modo=ok correlation=9f7be614d11c48df95f89f7a503e6072 traceparent=00-
1d28d3a6c40078362b6dc1324c3b5042-7825789c5dcdb667-01
[calendario] #2 GET /feriados/2026 modo=ok correlation=ef533df3629142e2849cc4a073055088 traceparent=00-
376cfe0e59d34ba121a5abad34806b1e-2f7c66c276495640-01
[calendario] #1 GET /feriados/2026 modo=ok correlation=23e760bb47614afb9fa007450e4c1b1b traceparent=00-
dfda6504de0c1e6d5ba192537dc044b8-85a7ab6e0dd3ddc7-01
```

Cache e rate limiting

```
### GET anônimo nº 1 de /calendario/feriados/2026
HTTP/2 200
### GET anônimo nº 2 de /calendario/feriados/2026
HTTP/2 200
### GET anônimo nº 3 de /calendario/feriados/2026
HTTP/2 200
### GET anônimo nº 4 de /calendario/feriados/2026
HTTP/2 200
### GET anônimo nº 5 de /calendario/feriados/2026
HTTP/2 200
Requisições que chegaram à dependência (5 chamadas anônimas): {"modo":"ok","recebidas":1}
### GET AUTENTICADO nº 1 (com cookie: não usa cache compartilhado)
HTTP/2 200
### GET AUTENTICADO nº 2 (com cookie: não usa cache compartilhado)
HTTP/2 200
Requisições que chegaram à dependência (2 chamadas autenticadas): {"modo":"ok","recebidas":2}
### Invalidação do cache pela tag 'feriados' (Administrador)
HTTP/2 204
### GET anônimo após invalidação
HTTP/2 200
Requisições que chegaram à dependência: {"modo":"ok","recebidas":1}

===== 6. Rate limiting (política 'publica' reduzida para 10 req / 10 s por IP nesta execução) =====
200 200 200 200 200 200 200 200 200 200 429 429 429 429
### Requisição rejeitada (detalhe)
$ curl https://localhost:7080/api/v2/calendario/feriados/2026
HTTP/2 429
content-type: application/problem+json
retry-after: 10
x-correlation-id: 016bbffa5b26441a9a70b132eab79855
{
    "title": "Muitas requisições.",
    "status": 429,
    "detail": "Aguarde o tempo indicado em Retry-After.",
    "instance": "GET /api/v2/calendario/feriados/2026",
    "traceId": "00-fd0dceef3ca21ef8179353fc2985df75-6ed952eb932bebd4-01",
```

Health checks

```
### Liveness com dependência saudável
HTTP/2 200
Healthy
### Readiness com dependência saudável
HTTP/2 200
Healthy
### Detalhes (Administrador)
HTTP/2 200
    "status": "Healthy",
            "nome": "banco",
            "status": "Healthy",
            "nome": "calendario",
            "status": "Healthy",
### Detalhes (anônimo)
HTTP/2 401
    "status": 401,
### Liveness com dependência INDISPONÍVEL (processo continua vivo)
HTTP/2 200
Healthy
### Readiness com dependência INDISPONÍVEL
```

```
HTTP/2 503
Unhealthy
### Detalhes (Administrador) com dependência indisponível
HTTP/2 503
    "status": "Unhealthy",
            "nome": "banco",
            "status": "Healthy",
            "nome": "calendario",
            "status": "Unhealthy",
```

Exportador de console: span customizado e métricas próprias

```
== Span customizado (ConsoleExporter) ==
Activity.TraceFlags:         Recorded
Activity.ParentSpanId:       181fa9752f2257a2
Activity.DisplayName:        tarefa.concluir
Activity.Kind:               Internal
Activity.StartTime:          2026-11-09T01:53:04.4056934Z
Activity.Duration:           00:00:00.0442466
Activity.Tags:
    tarefa.id: 6
    tarefa.concluida: true
Instrumentation scope (ActivitySource):
    Name: PortalTarefas
Resource associated with Activity:
    service.name: portal-tarefas-api
    service.version: dev
    service.instance.id: vm
    telemetry.sdk.name: opentelemetry

== Métrica portal.tarefas.criadas (ConsoleExporter) ==
Metric Name: portal.tarefas.criadas, Description: Tarefas cadastradas, Unit: {tarefa}, Metric Type: LongSum
Instrumentation scope (Meter):
    Name: PortalTarefas
(2026-11-09T01:53:04.2200686Z, 2026-11-09T01:53:17.1388005Z] prioridade: Alta
Value: 1

Metric Name: portal.tarefas.concluidas, Description: Tarefas concluídas, Unit: {tarefa}, Metric Type: LongSum
Instrumentation scope (Meter):
    Name: PortalTarefas
(2026-11-09T01:53:04.2201281Z, 2026-11-09T01:53:17.1388020Z]
Value: 1


== Métrica portal.tarefas.concluidas (ConsoleExporter) ==
Metric Name: portal.tarefas.concluidas, Description: Tarefas concluídas, Unit: {tarefa}, Metric Type: LongSum
Instrumentation scope (Meter):
    Name: PortalTarefas
(2026-11-09T01:53:04.2201281Z, 2026-11-09T01:53:17.1388020Z]
Value: 1

Metric Name: portal.dependencia.calendario.duracao, Description: Duração das chamadas ao serviço de calendário
(inclui tentativas), Unit: ms, Metric Type: Histogram
```

Carga controlada

```
===== 7. Carga controlada local (autocannon) — NÃO representa capacidade de produção =====
Máquina: 4 vCPU, contêiner de desenvolvimento; API em Debug; SQL Server no mesmo host.

$ npx autocannon -c 10 -d 10 https://localhost:7080/health/live
```

Stat	2.5%	50%	97.5%	99%	Avg	Stdev	Max
Latency	0 ms	0 ms	3 ms	4 ms	0.37 ms	0.95 ms	29 ms

Stat	1%	2.5%	50%	97.5%	Avg	Stdev	Min
Req/Sec	8,099	8,099	11,655	14,399	11,899.8	1,972.34	8,098
Bytes/Sec	2.34 MB	2.34 MB	3.37 MB	4.16 MB	3.44 MB	570 kB	2.34 MB

```
Req/Bytes counts sampled once per second.
# of samples: 10
119k requests in 10.03s, 34.4 MB read
```

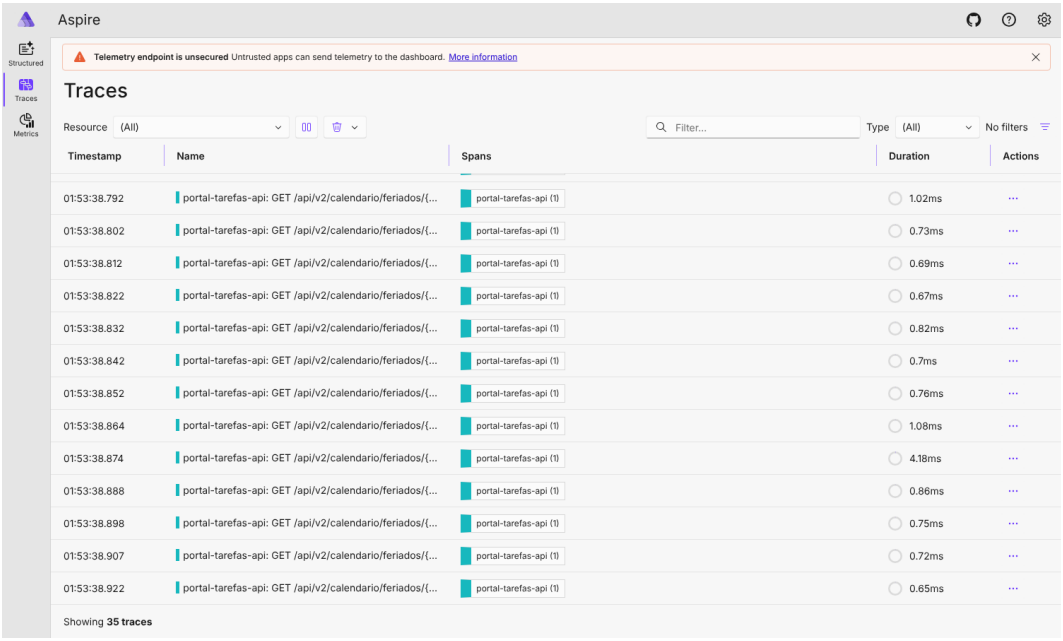


Figura B1 – Lista de traces no painel local

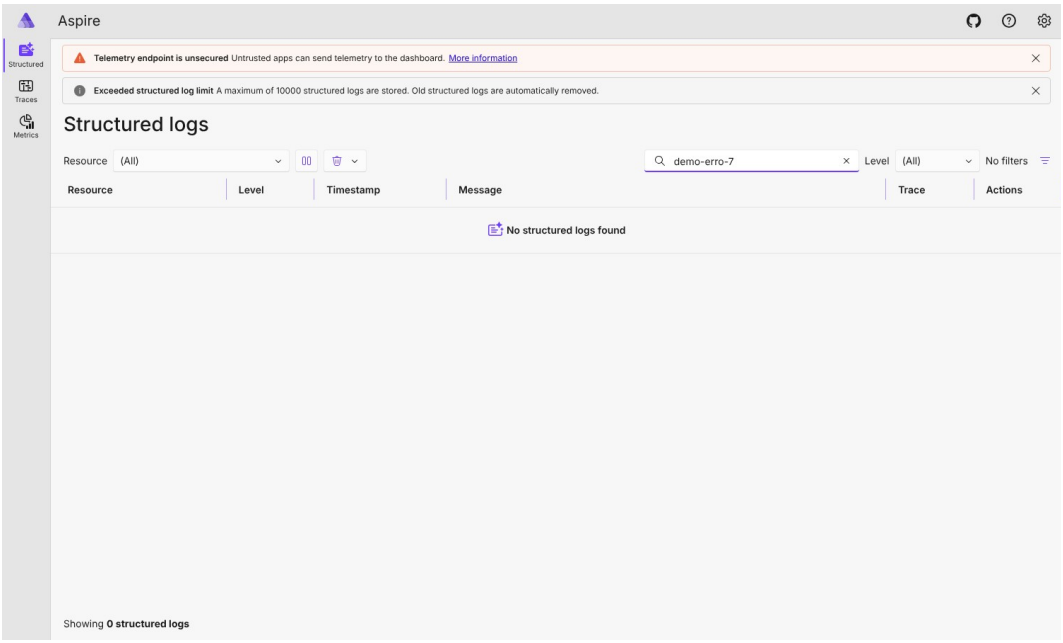


Figura B2 – Logs estruturados filtrados pelo identificador de correlação

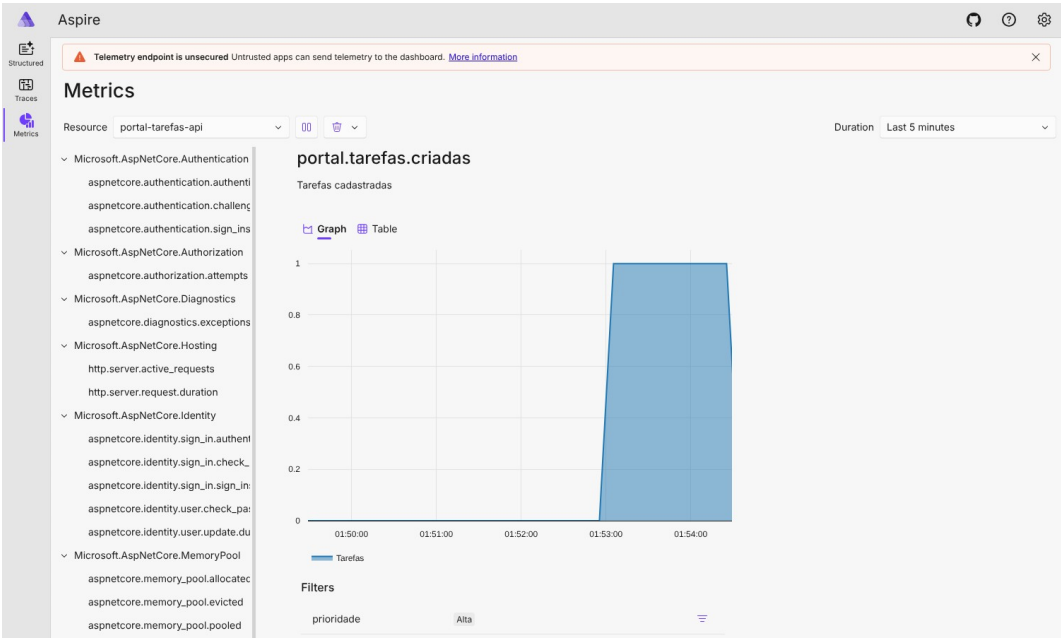


Figura B3 – Métrica de negócio portal.tarefas.criadas

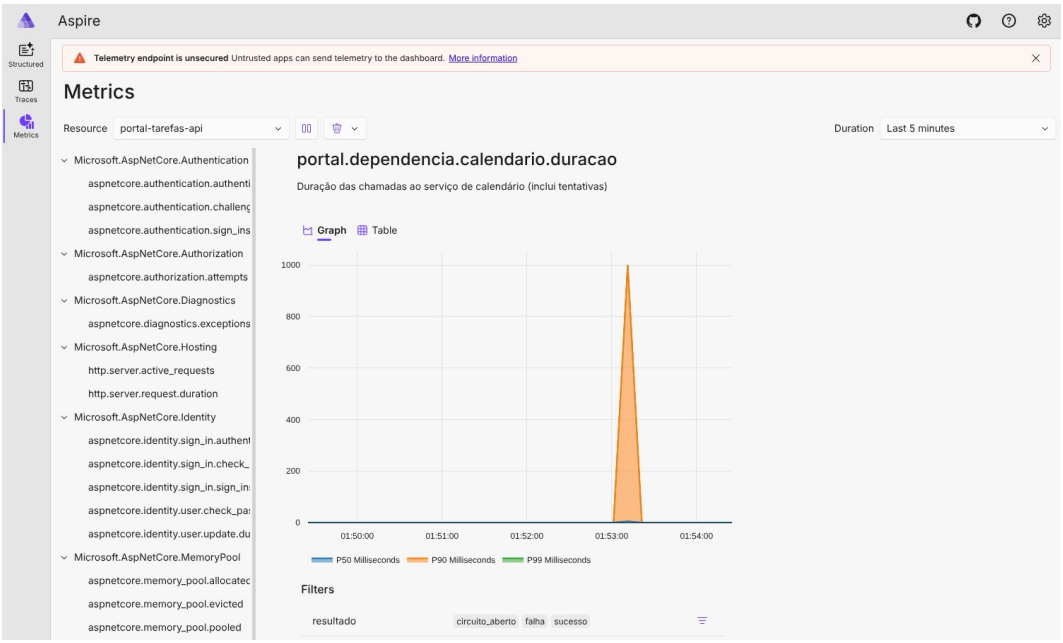


Figura B4 – Histograma de duração da dependência por resultado (sucesso, falha, circuito_aberto)